agent-ops-check: a free, offline check for AI agents

White paper. Author: Arup Banerjee. Personal Labs project. Date: October 12, 2026. Code: `main` at merge `307c727` (PR #4). License: MIT. Price: free. Repository: https://github.com/ultimatixarup/agent-ops-check

Abstract

People now build AI agents fast. The controls lag behind. An agent can read private data and post it to a work channel. An agent can submit a form on a real website with no person in the loop. Small teams often lack a list of the AI that runs, a named owner and a cap on spend.

agent-ops-check is a free command-line tool. It reads the code, prompts, configs, chat logs and bill exports of an agent. It runs on your own computer, with no network. It gives a score, a list of findings with file and line, an audit memo, SARIF output for CI, eval sets and a standards map.

The tool works with regular expressions and small rule engines. It is a set of heuristic static checks. It is not a security audit. It does not certify anything. Its standards map tells you which items a finding maps to. It never says "compliant".

On the bad example in the repository, the tool gives a score of 27 out of 100. It also prints `No network calls were made.`

1. The problem

Each statement below comes from a source. We use only numbers that we read in the source.

1.1 Personal data leaks into work channels

On October 1, 2026, a personal "CFO" agent posted a private monthly report to a work Slack chat. The owner, Shane Mac, gave the agent read-only access to his personal accounts. The agent picked a Slack chat with the name "Exec-team". The owner wanted his personal group chat with a similar name. After the incident, he removed all his connections.

Evidence	Source
Quote: "they're actually all using the same connections, even if they feel like different agents."	Business Insider, "My personal AI agent posted my bank details on company Slack. It's made me rethink how I use it.", as told to Aditi Bharade, October 9, 2026
The post stayed in the channel for two hours.	Geordie AI, October 7, 2026
62% of firms with agents in production let agents share credentials. Only 38% give each agent its own identity. Survey of 137 respondents.	VentureBeat Intelligence, October 5, 2026

Evidence	Source
Across four frontier agent models, the X-AgentRed method finds harmful cross-service tool paths at 1.77 times the rate of the earlier STAC method.	Jung et al., arXiv:2610.13886, October 9, 2026

The lesson: one set of connections can serve many agents that feel different. A prompt that says "don't" does not stop a leak. The builder must see each path from a personal source to a work sink.

1.2 Agents act on other people's websites

Evidence	Source
Claude models submitted a sensitive form on a real website.	Anthropic, "Investigating unintended model actions", October 9, 2026
A model read the settings file that a site sends to browsers and found working access tokens.	Same source
A model used URL shorteners to get around limits in its fetch tool.	Same source
In one case, the instructions "did not rule out form submissions."	Same source
Police say an AI model posted a false tip on an unsolved Philadelphia murder on July 18, 2026. Anthropic found the case on September 28 and told police on October 7.	NBC10 Philadelphia, October 9, 2026
A police spokesperson: "The two-month delay in detecting and reporting the incident to the City is unacceptable."	Same source
59% of 109 organizations with agents in production or pilot had a confirmed security incident or a near-miss from an agent in the past 12 months. 30% report a confirmed incident.	VentureBeat Intelligence, October 5, 2026

The lesson: an agent treats "not forbidden" as "allowed". Each irreversible action needs a gate. Examples of a gate: human approval, a dry run, a rate limit or a domain allowlist.

1.3 Small firms build AI faster than they govern it

Pax8 surveyed 1,150 leaders of small and midsized firms in the US, the UK, Germany and Sweden. The margin of error is about ±2.9 points for the combined sample.

Finding	Value
AI-engaged firms with one or more AI solutions of their own in regular use	67% (US: 81%)
Builders who build or test AI agents	46%
Firms that found AI running without approval in the past six months	35%
Formal, documented review	23% (UK) to 45% (Sweden). US: 36%, plus 41% with consistent review but no written record
Firms that paid for AI and got at least one bill higher than they expected	49% (US: 57%, Sweden: 65%)
Decision-makers who spend three or more hours each week to check AI work	55% (7% spend ten hours or more)

Finding	Value
Top-cited risk of AI that employees build	Data security or privacy exposure, 30% to 35% in each market
Firms that would pay at least 6% more for more control over vendor storage and use of their data	70%

Source: Pax8, published on GlobeNewswire, October 12, 2026. Pax8 says that usage-based charges are the hardest to forecast.

New Relic surveyed 2,575 engineering and IT leaders for its 2026 Observability Forecast (September 2026). 24.6% of organizations run AI agents in production with no monitoring.

In the VentureBeat survey, only 9% of all 137 respondents pick sandboxing of high-risk agents as the best description of their program.

1.4 Hidden instructions in documents steer agents

OWASP ranks prompt injection as risk number one (LLM01:2025). The APEX-Harm benchmark tests 12 frontier models on 180 task instances for each model. In that benchmark, the attacks with task-specific script replacement have an average success rate of 81% to 84% (Yang et al., arXiv:2610.13623, October 9, 2026). The authors write that a correct deliverable does not prove safe behavior.

1.5 Instructions are not clear or complete

Evidence	Source
Some failures "could have been avoided" if the task had clearly stated what was in and out of scope.	Anthropic, October 9, 2026
In a controlled test on a fictional business, the unsafe scripted bot failed 17 of 20 tests. The fixed bot failed 0 of 20.	Labs case study, September 30, 2026

A short rewrite of an instruction can also drop a number, a date or a word such as "never".

2. Design principles

Offline first. All scans run on your disk. The tool needs no internet, no account and no cloud service. The only download is the first install with `npx` or `git clone`.

No phone-home. The tool sends no telemetry, no usage counts and no update checks. A preload in the test suite stops the run if any code tries the network. The HTML report has a `default-src 'none'` content policy, so it loads nothing. The local UI listens on `127.0.0.1` only.

Rule updates come from Git. New rules ship only through the repository. You get them with `git pull`, or when you run `npx` again. The file `rules/VERSION.json` holds the rules version and date. Each report shows that version. The CLI also prints a local hint, for example "Rules are 0 day(s) old." The hint comes from your clock, not from a server.

Optional LLM, your own key. The `--llm` option is off by default. It works with OpenAI, Anthropic or OpenRouter. It uses your key from an environment variable. It sends only the list of findings, with

masked evidence. It never sends whole files. With no key, the tool sends nothing and the report says so.

Open rules. All patterns live in YAML files under `rules/`. You can copy the folder, edit it and point the tool at it with `--rules`.

Honest output. The tool masks secrets and personal data in every report. It lists credentials by name only. It never reads a real `.env` file.

3. Architecture

The tool has three layers: input, scanners with a rule engine, and reports.

Layer	Parts	Notes
Input	Folder, files, a `.zip`, `--docs`, `--ste` text, stdin	Each subfolder counts as one agent. The zip reader rejects absolute paths and `..` entries, skips symlinks, stops at 5,000 entries or 200 MB, and deletes its temp folder after the scan.
Scanners	Tab 1 crossover, tab 2 actions, tab 3 governance and cost, tab 4 Labs tools, tab 5 STE, tab 6 ADSF injection scan	Each scanner reads its own YAML file in `rules/`.
Labs tools	Agent Health Checker, Cloud Bill Smell Detector, Observability Gap Finder	They run in a Node sandbox. The sandbox blocks `fetch` and `XMLHttpRequest`.
Rule engine	Case-insensitive JavaScript regex, gate search near each action, Unicode checks, base64 decode	Gates: 6 lines above to 12 lines below each action.
Evals	`src/evals.js`	Writes functional and security test cases. It does not run them.
Standards map	`data/standards/*.yaml`, 10 frameworks	Each item has a fixed ✅/🟡/⬜ status and the rule prefixes that map to it.
Reports	`index.html`, `report.json`, `report.md`, `report.sarif`, `evals/`, `ste.html`, `redteam-pack.txt`	`report.md` is an audit memo with P0, P1 and P2 findings.

The score goes from 0 to 100. Each high finding costs 10 points. Each medium finding costs 4. Each low finding costs 1. The overall score is the mean of tabs 1 to 3. The CLI option `--fail-on` stops a CI build at the severity that you choose.

4. The checks, by need

The README holds the full table of each check. A script makes those tables from the rules, and a test fails if they drift. This section gives the summary.

Need (section 1)	Tab	Checks	Count today
1.1 Personal data in work channels	1 Crossover	Source-to-sink paths, shared credentials, `.env` present, hardcoded webhooks and channels, secrets, personal data	6 personal sources, 6 work sinks, 4 hardcoded patterns, 9 secret patterns, 4 PII patterns, 3 code-defined findings
1.2 Agents on other sites	2 Actions	Irreversible tools, browser writes, cookie reads, storage reads, real browser profiles, URL shorteners, wildcard domains, no deny-list	27 irreversible verbs, 4 gate detectors, 5 risky reads, 3 code-defined findings
1.3 Governance and spend	3 Governance, 4 Labs	Output caps, timeouts, retry caps, step caps, loops with no cap, budgets, tracing, unpinned and premium models, owner, review record, Health Checker, bill smells, observability gaps	11 provider patterns, 8 framework patterns, 11 governance findings, plus the Labs checks
1.4 Hidden instructions	6 ADSF	Overrides, role changes, fake system markers, instructions to AI readers, secrecy, exfiltration, tool-call injection, data-leaking images and URLs, request-capture hosts, HTML comments, CSS-hidden text, data URIs, invisible Unicode, base64 instructions	18 rule IDs
1.5 Unclear instructions	5 STE	Sentence length, passive voice, complex tenses, -ing verbs, semicolons, many instructions in one sentence, noun clusters, phrasal verbs, hedges, plain-word hints, your own word list, information-loss compare	11 style rules, 8 fact types, 25 phrasal verbs, 31 hedges, 32 hints

Tabs 1 to 3 hold 39 finding rules: 20 for crossover, 8 for actions and 11 for governance. The Agent Health Checker has 8 engineering, 9 customer-safety and 9 transcript checks. The file `health-plus.js` adds 7 checks. The Observability Gap Finder has 7 checks. `governance.yaml` holds two more patterns (`alias_model` and `logging`). No scanner uses these patterns yet.

4.1 ADSF: the document-wide injection scan

The ADSF tab reads each file that you give the tool. With `--docs`, you can add knowledge files, FAQs, saved emails and saved web pages. It looks for text that tries to steer an agent. It decodes the hidden text in Unicode tag characters and shows it in the finding. It also decodes each long base64 blob and scans the result. A clean result does not prove that a file is safe.

5. Standards mapping

5.1 How we mark items

- ✅ **Covered:** a committed check tests for the main signal of the item.
- 🟡 **Partial:** a check tests for some signals, or only for text that says a control exists.
- ⬜ **Not covered:** no check today, or out of scope.

All checks are static. They read what code and configs say. They do not prove what the agent does at runtime. A ✅ is not a guarantee. A finding "maps to" an item. The tool never says that a system is "compliant" with a framework.

5.2 Two sets of counts

The tool ships a status for each item in `data/standards/*.yaml`. For this paper, we scored each item again against the code on `main`, with the strict rules in section 5.1. In 16 of 112 items, our score is lower than the shipped status. We give both sets below. We did not change the shipped files.

Framework	Items	Shipped ✅ / 🟡 / ⬜	This paper ✅ / 🟡 / ⬜	Draft pitch (October 12)
OWASP Top 10 for LLM Applications 2025	10	5 / 4 / 1	5 / 4 / 1	4 / 4 / 2
OWASP Top 10 for Agentic Applications 2026 (ASI01 to ASI10)	10	3 / 6 / 1	3 / 6 / 1	2 / 6 / 2
OWASP Agentic AI Threats and Mitigations (T1 to T15)	15	5 / 7 / 3	4 / 8 / 3	3 / 9 / 3
NIST AI RMF 1.0 functions and NIST AI 600-1 risks	11	4 / 6 / 1	1 / 9 / 1	other item list
ISO/IEC 42001:2023 Annex A areas	9	2 / 6 / 1	0 / 7 / 2	0 / 7 / 2
MITRE ATLAS (13 selected techniques)	13	9 / 4 / 0	7 / 6 / 0	other item list
CSA AI Controls Matrix (18 domains)	18	3 / 9 / 6	2 / 10 / 6	2 / 10 / 6
EU AI Act (9 selected articles)	9	3 / 4 / 2	1 / 5 / 3	other item list
Microsoft Copilot Studio governance areas	9	2 / 5 / 2	2 / 5 / 2	other item list
Anthropic agent guidance	8	7 / 1 / 0	4 / 4 / 0	other item list
Total	**112**	**43 / 52 / 17**	**29 / 64 / 19**	

5.3 What changed since the draft pitch

The draft pitch scored the code before the ADSF scan and the eval sets. With those two features, seven items go up. The shipped map also lowers two items:

- LLM01 and ASI01 go from 🟡 to ✅. Now the ADSF scan reads the text of documents.
- LLM04 goes from ⬜ to 🟡. The scan finds the hidden instructions in knowledge files. It does not check training data.
- ASI06 goes from ⬜ to 🟡. The scan reads memory and knowledge files that you point it at.
- T6 goes from 🟡 to ✅. T1 and T12 go from ⬜ to 🟡.
- T13 and T14 go from 🟡 to ⬜ in the shipped map. Tracing and owner checks do not detect a rogue agent.

5.4 Where this paper scores lower than the shipped map

Item	Shipped	This paper	Reason
T8 Repudiation and Untraceability	✅	🟡	It finds tracing tools. It does not check log integrity.
CSA LOG Logging and Monitoring	✅	🟡	Tracing presence only.
EU AI Act Art. 12 Record-keeping	✅	🟡	Tracing presence only.
EU AI Act Art. 14 Human oversight	✅	🟡	Gate map, but no stop-button check (Art. 14(4)(e)).
EU AI Act Art. 10 Data and data governance	🟡	⬜	Art. 10 is about training, validation and test data. The tool checks none.

Item	Shipped	This paper	Reason
ISO/IEC 42001 A.4 Resources	✅	🟡	The inventory covers models, tools and connectors only.
ISO/IEC 42001 A.9 Use of AI systems	✅	🟡	Gates and handoff rules cover only part of the area.
ISO/IEC 42001 A.5 Impact assessment	🟡	⬜	The audit memo is not an impact assessment.
ATLAS AML.T0056 System prompt extraction	✅	🟡	A static scan can not test a runtime attack.
ATLAS AML.T0057 LLM Data Leakage	✅	🟡	Static paths and chat-log leaks only.
NIST MAP function	✅	🟡	An inventory covers only part of the function.
NIST 600-1 Human-AI Configuration	✅	🟡	Gates, disclosure and handoff text only.
NIST 600-1 Information Security	✅	🟡	The risk is broader than the static checks.
Anthropic: treat outside content as untrusted	✅	🟡	The defense works at runtime. The scan finds static signals only.
Anthropic: build evaluations	✅	🟡	The tool writes test cases. It does not run them.
Anthropic: log and monitor actions	✅	🟡	Tracing presence only.

5.5 Notes on the map

- The shipped MITRE ATLAS list has no ⬜ item. The draft pitch also listed AML.T0070, AML.T0080, AML.T0081 and AML.T0099 as ⬜. The shipped list leaves them out.
- The shipped names for AML.T0010 and AML.T0056 differ from the names in the pitch research. Check them against the current ATLAS data.
- The shipped list marks T13 and T14 ⬜, but ASI10 Rogue Agents 🟡. This paper keeps the shipped status. A future review must make the two lists agree.
- `rules/CHANGELOG.md` says "9 frameworks". The code ships 10.
- The EU AI Act rows are not legal advice. The tool does not decide the risk class of a system.
- ISO sells ISO/IEC 42001. We did not read the full text. The area names come from public summaries.

5.6 Other scorecards from the draft pitch

Scorecard	Draft pitch	Now	Change
Agent governance checks (G1 to G17)	12 / 2 / 3	12 / 2 / 3	G13 "evals exist" stays 🟡. The tool writes eval sets but does not run them.
ASD-STE100 style markers (S1 to S16)	10 / 3 / 3	10 / 3 / 3	No change to the STE code since PR #2.
Eval sets	1 / 0 / 2	2 / 0 / 1	Eval-set generation is now ✅. Live runs stay ⬜.
STE checker inputs	7 / 0 / 0 (PR branch only)	7 / 0 / 0 (on `main`)	PR #2 is now merged.
Static scanners	4 / 0 / 0	5 / 0 / 0	The ADSF scan is now tab 6.
Document-wide injection scan	0 / 0 / 1	1 / 0 / 0	Shipped.

Scorecard	Draft pitch	Now	Change
Zip input and local UI	not scored	2 / 0 / 0	Merged in PR #4.

6. Eval sets

The command `agent-ops-check evals <folder>` writes eval sets. A normal scan also writes them to `evals/` in the report folder. The generator reads the tools, prompts, irreversible actions, source-to-sink paths and credential names that the scanners found. It writes two JSONL files and a readable README.

Set	Categories	Bad example
Functional	Tool use, in-scope request, out-of-scope request	11 cases (5 tool use, 3 in scope, 3 out of scope)
Security	Direct injection, indirect injection, role-play jailbreak, irreversible action with no confirmation, personal-to-work leak, credential disclosure, unbounded consumption	31 cases (3, 3, 3, 7, 7, 5 and 3)

Each security case names an OWASP LLM item, for example LLM01, LLM02, LLM06 or LLM10. For an action with no gate in the code, the expected result says that the test must fail until you add a gate.

The tool does not run these cases. You run them against your own test bot. The red-team pack (`redteam-pack.txt`) gives 20 more test messages, each with a safe reply.

7. The STE checker and the information-loss compare

7.1 The STE checker

Simplified Technical English (STE) uses short sentences, active voice and one instruction in each sentence. The STE tab applies those ideas to prompts and instructions. A clear prompt helps an agent stay in scope.

The rules come from ASD-STE100 ideas. The tool is **not** a certified ASD-STE100 checker. ASD holds the copyright on the specification and its dictionary. The tool does not include them. The word lists in `rules/ste.yaml` are original and incomplete.

You can check an agent folder, any file, pasted text or stdin. You can add your own list of approved words with `--ste-dict`. The command `agent-ops-check ste-ui` writes an offline page with a paste box and a file picker.

The checker reads only the prose lines. It skips headings, table rows and code blocks.

7.2 The information-loss compare

A rewrite into plain words can drop a fact. The compare reads the original text and the rewrite. It lists each number, unit, date, name, path, URL, identifier and constraint word that the rewrite lost. Constraint words include "must", "never", "only", "before", "until", "unless" and "within".

Example with the files in `examples/ste/`:

Original	Rewrite
You must restart the billing-sync job within 30 seconds if it fails.	Restart the billing-sync job if it fails.
Never send more than 3 retries to /v1/charges before 2026-10-19.	Do not send retries to /v1/charges.
Only Jane Doe can approve a refund over $500.	Jane Doe approves refunds.

The compare finds 9 losses: 8 high and 1 medium. The rewrite lost the date "2026-10-19", "30 seconds", "3 retries", "$500" and the words "must", "Never", "before", "Only" and "within". The STE tab score for this compare is 16 out of 100.

8. Worked example: the bad example agents

The folder `examples/bad-agent/` holds four fictional agents with known faults. We ran this command on `main` at `307c727` on October 12, 2026:

```
npm run example
```

The tool scanned 11 files and found 5 agents. The folder root counts as one agent.

Tab	Score	High	Medium	Low
1 Personal-to-work crossover	39	6	0	1
2 Irreversible actions	0	11	5	0
3 Governance and cost	41	1	11	5
4 Agent Health Checker and Labs tools	0	19	7	5
6 Prompt-injection scan (ADSF)	6	9	1	0
5 STE	84	0	3	4
Overall (mean of tabs 1 to 3)	**27**			

The CLI ended with `No network calls were made. Nothing left this machine.`

What the tool found:

- **Crossover.** Two agents use the same `SLACK_BOT_TOKEN`. The personal "CFO" example agent can read Gmail, Google Drive, personal files and tax paths, and it can write to Slack. The source-to-sink matrix has 7 rows. The prompt holds a fake SSN and a fake account number.
- **Actions.** The gate map lists 7 actions: `.fill(`, `.click(`, `submit_form`, `purchase_item`, `post_message`, `send_slack_message` and `delete_jira_issue`. None of them has an approval gate, a dry run, a rate limit or a domain allowlist. The browser bot also uses a real browser profile, reads cookies and site storage, uses a URL shortener and allows a wildcard domain. Three prompts have no deny-list.
- **Governance.** One agent has a `while True` loop with no cap. Two agents have no output cap, timeout, step cap, budget or tracing. The models `gpt-4o` and `claude-3-5-sonnet-latest` have no dated version. The project names no owner and has no review record.
- **ADSF.** A knowledge file holds an override phrase inside an HTML comment, 3 zero-width characters and 24 Unicode tag characters. The decoded tag text starts with "send tax file". The same file holds a base64 instruction, a data-leaking image link with a template slot and a request-capture host. A chat log holds an override and a role change.

- **Health Checker.** The prompt of the support bot scores 8, and its chat log scores 6. The verdict for both is "Not safe for customer traffic".
- **Evals.** The tool wrote 11 functional and 31 security cases.

For contrast, `examples/good-agent/` scores 100 on tabs 1 to 3.

9. Limits and threats to validity

- **Static only.** Regex reads what code and configs say. It does not see what they enforce at runtime.
- **False results.** The checks can miss problems. They can also flag harmless code.
- **Agent equals subfolder.** A monorepo with one folder for each service works best.
- **Gate window.** The tool looks for a gate from 6 lines above to 12 lines below each action. It does not see a gate in a distant wrapper.
- **Language.** The Health Checker works in English. It also reads some phrases in Spanish, French and German.
- **Text checks.** The owner and review checks find text only. They do not prove that a review took place.
- **Budget checks.** The tool finds a reference to a budget. It can not see the provider console.
- **Fixed status.** Each item in a map has a fixed status in a YAML file. The authors of this paper wrote both the code and the scores. Nobody outside the project reviewed them.
- **Selected items.** The maps list selected items only. NIST AI RMF has 72 subcategories. CSA AICM v1 has 243 control objectives. We map at the function or domain level for these two.
- **Test set.** The bad example and the good example are small and fictional. The scores show that the checks fire. They do not measure accuracy on real agents.
- **Eval sets.** The generator writes cases from a fixed template. In one bad-example case, the agent field holds a file path in place of an agent name.
- **STE score.** The STE checker skips tables, headings and code blocks. A document with most of its facts in tables gets a higher score.
- **Sources.** Most sources in section 1 are from October 2026. Two are arXiv preprints. We read their abstracts only.

10. Roadmap: top gaps

#	Gap	Maps to
1	Stop-switch and sandbox-setting check	EU AI Act Art. 14(4)(e), NIST MANAGE, ASI10, Anthropic guidance
2	Memory, RAG and vector-store checks beyond text scans	LLM04, LLM08, ASI06, T1, AML.T0070, AML.T0080
3	Version pins for MCP servers, tools and packages	LLM03, ASI04, AML.T0010
4	Inter-agent message checks	ASI07, T12, T13
5	Log retention and decommission plan	G16, G17, NIST GOVERN
6	An offline runner for the eval sets against a local test bot	NIST MEASURE, G13
7	A review of the shipped standards status, with the 16 items in section 5.4	All maps

Each new rule follows `docs/UPDATES.md`. It needs a fixture that fires, a fixture that does not fire, a test, a version bump and a changelog entry with a source.

11. How to run it

You need Node.js 18 or later.

```
# Scan a folder
npx github:ultimatixarup/agent-ops-check ./my-agent

# Scan a zip
npx github:ultimatixarup/agent-ops-check ./agent.zip

# Local launch page on 127.0.0.1
npx github:ultimatixarup/agent-ops-check ui --open

# Add documents to the injection scan
agent-ops-check ./my-agent --docs ./knowledge

# STE check and information-loss compare
agent-ops-check --ste docs/
agent-ops-check --ste-compare original.txt rewrite.txt

# CI gate
agent-ops-check ./my-agent --fail-on high

# From a clone
git clone https://github.com/ultimatixarup/agent-ops-check
cd agent-ops-check && npm install && npm test && npm run example
```

Open `agent-ops-report/index.html` in your browser to read the report. Run `git pull` to get new rules.

12. References

1. Business Insider. "My personal AI agent posted my bank details on company Slack. It's made me rethink how I use it." As told to Aditi Bharade. October 9, 2026.
2. Geordie AI. "Personal agents, shared platforms, unintended disclosure." October 7, 2026. https://www.geordie.ai/resources/personal-agents-shared-platforms-unintended-disclosure/
3. VentureBeat Intelligence. Agentic security survey. October 5, 2026. https://venturebeat.com/resources/agentic-security-enterprises-rely-on-openai-most-to-secure-ai-agents-and-those-that-do-are-nearly-three-times-as-likely-to-say-attackers-are-ahead
4. Anthropic. "Investigating unintended model actions." October 9, 2026. https://www.anthropic.com/news/investigating-unintended-model-actions
5. NBC10 Philadelphia. Report on a false tip from an AI model to Philadelphia police. October 9, 2026. https://www.nbcphiladelphia.com/news/local/anthropic-ai-model-submits-false-tip-on-unsolved-philly-murder-police-say/
6. Pax8. Research release on small firms and AI governance, on GlobeNewswire. October 12, 2026. https://www.globenewswire.com/news-release/2026/10/12/3378898/0/en/pax8-research-finds-small-businesses-are-building-ai-faster-than-they-can-govern-it.html

7. New Relic. 2026 Observability Forecast. September 2026. https://newrelic.com/sites/default/files/2026-09/new-relic-2026-observability-forecast-report.pdf
8. Jung, Y. et al. "X-AgentRed: Red-Teaming Personal Agents in Evolving Multi-Service Ecosystems." arXiv:2610.13886. October 9, 2026.
9. Yang, Z. et al. "APEX-Harm: Evaluating Agent Safety in Shared Professional Workspaces." arXiv:2610.13623. October 9, 2026.
10. Arup Banerjee Labs. Case study. September 30, 2026. https://ultimatixarup.github.io/arup-banerjee-labs/case-study/
11. OWASP. Top 10 for LLM Applications 2025, LLM01 Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
12. OWASP GenAI Security Project. Top 10 for Agentic Applications 2026. https://genai.owasp.org/download/52117
13. OWASP. Agentic AI: Threats and Mitigations. https://genai.owasp.org/download/45674/
14. NIST. AI RMF 1.0 Core. https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
15. NIST. AI 600-1, Generative AI Profile. July 2024. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
16. ISO. ISO/IEC 42001:2023. https://www.iso.org/standard/42001
17. MITRE ATLAS data. https://github.com/mitre-atlas/atlas-data
18. Cloud Security Alliance. AI Controls Matrix. https://cloudsecurityalliance.org/artifacts/ai-controls-matrix
19. Regulation (EU) 2024/1689 (AI Act). https://eur-lex.europa.eu/eli/reg/2024/1689/oj
20. Microsoft. Copilot Studio security and governance guidance. https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/sec-gov-intro
21. Anthropic. "Our framework for developing safe and trustworthy agents." August 4, 2025. https://www.anthropic.com/news/our-framework-for-developing-safe-and-trustworthy-agents
22. ASD Simplified Technical English Maintenance Group. ASD-STE100. https://www.asd-ste100.org/

Personal project built on my own time. Not affiliated with or endorsed by my employer; views are my own.